

Code-Layout, Readability and Reusability

Date	02 OCTOBER 2026
Team ID	6ab7f7bfd083b0e2ee72b13b
Project Name	LegalEase: AI-Powered Legal Document Generator
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent Python formatting and indentation are used across backend and frontend modules.
2	Proper File Structure	Files and folders are logically organized	Yes	he project is logically separated into backend, frontend, ai_core, services, assets, scripts, and tests.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables and configuration names clearly represent their purpose, such as document_type, parties, terms, and GEMINI_API_KEY.
4	Function / Method Names	Functions are descriptively named	Yes	Functions are descriptively named, such as format_docx, format_pdf, format_txt, and generate_content.
5	Code Comments	Inline and block comments explain logic	Yes	Important modules and configuration files contain explanations of their purpose and implementation.
6	Modular Design	Code is split into reusable functions/modules	Yes	AI generation, validation, document processing, preview, branding, and exporting are

				separated into reusable modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Common document parsing and formatting logic is centralized in reusable services rather than duplicated across exporters.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Validation errors, AI failures, timeouts, rate limits, invalid input, and export failures are handled with user-friendly error responses.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	document_service.py	Python	TXT, DOCX, PDF and preview processing	High

2	gemini_generator.py	Python / Google Gen AI SDK	AI document generation	High
3	document_types.py	Python	Multiple legal document templates	High
4	models.py	Python / Pydantic	API request and response validation	High
5	export_service.py	Central export handling for TXT/DOCX/PDF	Python	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5/5	Clear separation of frontend, backend, AI logic, services, assets, and tests.
Readability	5/5	Meaningful module and function names with organized configuration and service responsibilities.

Reusability	5/5	Shared document-processing and export components reduce duplication and support future extensions.
Documentation / Comments	4/5	README and project structure provide substantial documentation; individual code comments can be expanded where necessary.
Overall Score	5/5	Well-structured, modular codebase suitable for maintenance and future feature development.